

내 일이 뭔지 아무도 모른다

비개발자가 자기 업무 흐름을 적으면 다이어그램이 자동으로 그려지고, 어디를
자동화 — 혹은 제거 — 할지 드러나는 도구.

1	한 문장	3
	이 제품이 파는 것	
2	비어 있는 자리	4
	왜 아무도 이걸 안 만들었나	
3	어떻게 동작하는가	5
4	설계 결정 여덟	6
	그리고 그 근거	
5	이 제품이 죽는 방법	9
6	시장과 경쟁	11
7	무엇을 어떤 순서로 만드나	12
8	지금 상태 — 정직하게	14

한 문장

01 · 이 제품이 파는 것

직원은 "설명하지 않아도 되는 업무 문서"를 얻고,
회사는 "직원이 직접 만든 자동화 백로그"를 얻는다.

같은 데이터로 두 개의 서로 다른 약속을 한다. 이 이중성이 제품 구조의 전부이고, 이 문서 나머지는 대부분 그 둘이 충돌하지 않게 만드는 방법에 관한 것이다.

직원에게 파는 것은 자동화가 아니다. 그건 회사에 파는 것이고, 직원에게는 다른 것을 준다 — 같은 걸 다섯 번 설명하지 않아도 되는 문서, 휴가 갈 때 대신할 사람에게 건넬 안내, 인수인계 때 그대로 쓰는 자료.

왜 이 구분이 중요한가

직원 화면에 "자동화"라는 단어가 뜨는 순간, 사용자는 자기 일을 없애는 문서에 성실히 데이터를 입력해야 하는 상황에 놓인다. 그러면 소요시간은 부풀려지고, 불편한 단계는 정치적으로 선택된다. 채택 문제가 아니라 데이터 품질 문제다.

비어 있는 자리

02 · 왜 아무도 이걸 안 만들었나

n8n이나 Zapier 같은 자동화 도구를 도입한 조직은 곧 같은 벽에 부딪힌다. 무엇을 자동화할지 아는 사람이 없다. IT팀은 시스템 흐름은 알아도 현업이 매주 세 시간씩 돌리는 엑셀 수작업은 모르고, 현업은 그게 자동화 대상이라는 생각 자체를 해본 적이 없다.

이 질문에 답하려는 기존 해법은 셋인데, 셋 다 같은 곳을 못 본다.

해법	보는 것	못 보는 것
프로세스 마이닝 Celonis · 퍼즐데이터	ERP·그룹웨어 이벤트 로그	로그가 남지 않는 업무. 그리고 국내 진입 패키지가 1.5억 원대다
태스크 마이닝 Scribe · StepHow	브라우저 안의 클릭과 입력	브라우저 밖 전부 — 전화, 오프라인 승인, 카톡, 부서 간 대기
다이아그램 툴 Miro · Whimsical	사람이 그린 그림	그림만 남고 데이터가 안 쌓인다. 집계도 비교도 불가능
컨설팅	인터뷰로 캐낸 실제 흐름	1회성. 프로젝트가 끝나는 날이 문서 사망일이다

공통점이 보인다. 로그에도 화면에도 남지 않는 업무가 통째로 사각지대다. 그런데 실무 시간의 상당 부분이 정확히 거기 있다 — 카톡 확인, 엑셀 수기 정리, 유선 승인, 옆 부서 회신 대기.

이 영역의 유일한 데이터 소스는 당사자의 기억이다. 그리고 그걸 구조화해서 수집하는 제품은 사실상 없다.

단, 문제의 주인을 혼동하면 안 된다

"내 일이 뭔지부터 알아야 한다"는 문제의식의 주인은 자동화를 도입하려는 쪽이다. COO, DX 담당, 경영지원실장. 영업팀 김대리가 아니다.

김대리는 자기 일을 안다 — 몸으로. 문서로 없을 뿐이다.

그에게 "당신 일이 뭔지 알아야 합니다"는 문제 제기가 아니라 잔소리다.

이 구분을 놓치면 제품 전체가 어긋난다. 그래서 §1의 이중 약속이 필요했고, 그게 이 기획의 첫 번째 결정이다.

어떻게 동작하는가

03

핵심 베틱은 하나다. 입력은 글처럼, 출력은 그림으로. 사용자는 선을 단 한 번도 직접 긋지 않는다.

비개발자에게 빈 캔버스와 노드 팔레트를 주면 얼어붙는다. Miro가 좋은 도구인데도 업무 정리에 잘 안 쓰이는 이유가 그것이다. 반면 사람들이 실제로 말하는 방식은 "메일이 오면 확인하고, 단순한 거면 바로 답장하고, 복잡하면 팀장님께 여쭙보고..." — 선형 서술에 가끔의 분기다.

- ① 붙여넣기
기존 워드 매뉴얼 · 카톡 인수인계 · 메일 스레드를 그대로 붙여넣으면
한국어 연결어미와 번호·동사 패턴으로 단계를 분해한다. AI 없이.
↓ 붙여넣을 게 없으면
- ② 질문 연쇄
한 번에 한 질문. 계층 없음, 실패 불가능.
"먼저 무슨 일이 시작되나요?" → "그다음엔요?" → 다섯 번
↓ 자동 전환
- ③ 아웃라인 + 다이어그램
왼쪽에 글을 적으면 오른쪽에 그림이 자동으로 그려진다
↓ 다 쓰고 나서
- ④ 메타데이터 카드
전체화면으로 한 화면 한 질문. "모르겠어요"가 정답과 같은 크기
↓
- ⑤ 인수인계 문서 · 요약 · 자동화 후보

사용자가 통과하는 다섯 단계. ①과 ②는 대안 관계다.

왜 아웃라인이 첫 화면이 아닌가

초안에서 가장 크게 뒤집힌 지점이다. 처음에는 "아웃라인 우선"이 핵심 베틱이었는데, UX 검토에서 정면으로 반박당했다.

사람은 자기 업무를 순서가 매겨진 계층 구조로 기억하지 않는다. 에피소드로 기억한다 — "메일이 오면 짜증나고, 엑셀에 옮기는 게 제일 오래 걸리고, 팀장님이 늦게 답하면 하루가 밀려요". 아웃라인은 이 원재료를 정리한 결과물인데, 첫 화면부터 정리된 형태를 요구하면 기억 인출과 구조 설계를 동시에 시키는 셈이다.

그래서 아웃라인은 첫 입력 표면이 아니라 편집기로 강등됐다. 질문 연쇄로 대여섯 개를 모은 뒤 "이렇게 정리해봤어요"라며 넘긴다.

설계 결정 여덟

04 · 그리고 그 근거

중요도 순이다. 첫 번째가 나머지 전부를 지배한다. 각 결정에는 뒤집는 조건이 붙어 있다 — 어떤 신호가 오면 재논의할지 미리 적어두지 않으면, 무엇을 가정하고 있는지 스스로도 모르게 된다.

1 · 관리자에게 개인 데이터를 보여주지 않는다

이 제품은 직원이 자발적으로 진실을 적어야만 작동한다. 부풀린 소요시간과 이상적인 해피패스로 채워진 데이터는 쓸모가 없다 — 자동화 후보 목록이 쓰레기가 되고, 그러면 경영진에게 한 약속도 못 지킨다.

그래서 관리자 화면에는 사람이 등장하지 않는다. 집계 단위는 프로세스이고, 5인 미만 그룹은 표시 자체가 차단된다.

진실을 얻는 유일한 방법은 그 진실이 자신에게 불리하게 쓰이지 않는다는 확신이고, 그 확신은 처리방침 문구가 아니라 **"만들 수 있는데 안 만들었다"**는 증명에서 나온다. 경영진은 "부서별로 좀 보면 안 되나"를 **6개월 안에 반드시 요구한다**. 그때 거절할 근거를 지금 만들어 두는 것이 이 프로젝트에서 할 수 있는 가장 가치 있는 일이다.

2 · Tab 들여쓰기를 분기 문법에서 뺀다

Tab으로 계층을 만드는 개념을 즉시 이해하지 못할 사용자가 **30~50%**로 추정된다. 대신 "만약 ~라면 / 아니면 ~라면"이라는 자연어 껍데기를 UI에 인쇄한다.

더 나쁜 건 실패 모드가 **조용하다**는 것이다. Tab을 안 눌러도 화면은 멀쩡히 그려지고, 사용자는 자기가 틀렸다는 걸 모른 채 나중에 "이 도구 이상하다"고 결론낸다. 시끄러운 실패는 고칠 수 있지만 조용한 실패는 신뢰를 갉아먹는다.

3 · 노드 타입은 세 개, 표현력은 속성으로

작업 · 분기 · 기다림. 타입을 늘리면 학습 부담이 늘고, 안 늘리면 실무를 못 그린다. 문법이 아니라 속성으로 흡수하면 커버리지가 55~65%에서 **85%**로 오른다.

병렬(AND)과 배타(XOR)를 구분하지 않으면 시각화만 어긋나는 게 아니라 **숫자가 틀린다**. 실측: 같은 그림에서 AND 4시간, XOR 2.5시간, 미구분 합산 5시간 — 각각 25%·100% 과대 계산.

4 · 작성 중에는 메타데이터를 묻지 않는다

작성 흐름에 여섯 필드를 노출하면 담당자 외에는 **20% 미만** 채워진다. 다 쓰고 나서 전체화면 카드로 분리하고, 입력이 아니라 **확인**으로 만든다 — "엑셀에 정리"라고 쓰면 줄 끝에 배지가 뜨고 클릭 한 번.

게으름이 아니라 세 가지다 — 인지 전환 비용(기억을 되짚는 중에 정량화 질문이 끼어든다), 정확도 불안 ("30분인지 40분인지 모르겠는데"), 보상 부재.

5 · 자기보고 데이터의 왜곡을 설계로 보정한다

빈도 왜곡이 시간 왜곡보다 치명적이다 — 시간 × 빈도 곱셈이라 오차가 곱해져 랭킹 상위가 통째로 뒤집힌다. 그래서 "평소 얼마나 자주"가 아니라 "**지난 7일 동안 몇 번**"으로 회상 기간을 고정한다.

가장 강력한 장치는 **총량 정산**이다. 그 사람이 적은 모든 흐름의 월 환산 합계를 되돌려준다 — "**합계 240시간 / 근무 160시간**". 그러면 본인이 재배분한다. 개별 추정을 백 번 다듬는 것보다 효과가 크다.

6 · 자동화보다 제거가 먼저

랭킹 이전에 묻는다 — "**자동화하기 전에 없앨 수 있는가?**" 승인 3단계 중 반려율 0%인 단계는 자동화 대상이 아니라 제거 대상이다.

제거 후보 리스트가 자동화 후보 리스트보다 훨씬 **희소하고 경영진에게 잘 팔린다**. 그리고 개발이 0줄이다. 실제 시드 데이터에서 나온 것 — 구매 팀장 결재(10~300만원)는 6개월 반려 0건에 평균 대기 3일. 기준선을 올리면 **리드타임 -3일, 연 62시간 회수**.

7 · 상대에 대한 평가를 수집하지 않는다

전 직원이 각자 적으면 "구매 요청" 하나가 요청자·팀장·구매팀·재무팀 네 조각으로 흩어진다. 이걸 잇고 **불일치를 발견**하는 것이 개인 도구로는 만들 수 없는 가치다 — "요청자는 2일 걸린다 하고 구매팀은 5일이라 한다."

단, 수집하는 것은 자기 관점의 사실뿐이다. 불일치는 두 사람의 자기 기술을 시스템이 대조해서 찾아낸다.

"**저쪽이 얼마나 늦나요**"를 한 번이라도 물으면, 이후 어떤 노출 정책으로도 정치를 막을 수 없다. 정치를 막는 건 노출 통제가 아니라 질문 설계다.

8 · 디자인은 마감이 아니라 기능

이유가 둘이다. "회사가 시킨 시스템"처럼 보이면 그 순간 끝난다 — 감시 도구 낙인은 카피가 아니라 시각적 톤에서 먼저 결정된다. 그리고 결과물이 남에게 공유된다 — 예뻐야 공유하고, 공유돼야 확산되므로 디자인이 곧 바이럴 계수다.

구체적 금지: % 게이지·도넛·진행률 바·리더보드·팀 평균(집계는 그리지 않는다), 신호등 색(색은 판단하지 않고 분류만), confetti(실직 공포를 정확히 자극한다), 파랑("회사 시스템" 인상).

이 제품이 죽는 방법

05

기능이 부족해서가 아니다. 위험한 순서대로 적는다.

사망 원인

방어

1. 효용 미증명

직원이 불편함을 적었는데 6개월간 아무것도 안 바뀌면, 그 사람은 물론 그가 속한 부서 전체가 두 번 다시 안 쓴다. → 자동화 실행 예산과 담당 인력 없이 오픈하지 않는다. 도구보다 "발굴된 것을 처리하는 팀"이 먼저다.

2. 감시 도구 낙인

한 번 찍히면 정책을 바뀌도 신뢰는 안 돌아온다. → 설계 결정 1번 전체가 이 방어에 쓰인다.

3. 입력 절벽

첫 문서를 끝까지 쓰는 사람이 20% 미만. → 시드 콘텐츠를 팀이 먼저 채우고, 붙여넣기 파싱과 질문 연쇄로 백지를 없앤다.

4. 죽은 문서 무덤

읽을 이유 없는 문서는 반드시 썬다. 실행에 쓰이는 문서만 산다. → 작성한 흐름을 체크리스트로 다시 열게 만든다. 여는 순간 틀린 단계가 그 자리에서 고쳐진다.

5. 레이아웃 점프

타이핑할 때마다 그림이 춤추면 즉시 신뢰를 잃는다. → 타이핑 중 재배포 금지, 구조 변경 시에만 다시 그린다.

단 하나의 생존 지표

D14 재편집률이 5% 아래면 이 제품은 "죽은 문서 생성기"다. 문서를 만들었다는 것만으로는 아무것도 증명되지 않는다. 2주 뒤에 다시 열어서 고치는 사람이 있어야 살아 있는 것이다.

측정하는 것

북극성 지표는 부서 커버리지다 — 최근 30일 내 프로세스를 1개 이상 완성했고, 그 문서가 타인에게 1회 이상 열람된 직원의 비율. 만든 것만으로는 세지 않는다.

지표	목표	못 맞추면
첫 다이어그램까지	≤ 8분	입력 절벽
작성 완료율	60%	결과물이 기대에 못 미침
노드 드래그 시도	< 1회/세션	레이아웃 신뢰 붕괴
공유율	30%	"보여줄 만하다"에 실패
열람자 / 작성자	≥ 3:1	성장 루프 없음
D14 재편집률	25%	죽은 문서 생성기

현실적 예측은 전사 40%가 최소 1개 문서, 그중 20%만 살아있는 문서다. 그 정도로 충분하다 — 자동화 후보 발굴은 전수조사가 필요 없고, 40%면 반복업무 지형도의 80%가 드러난다.

시장과 경쟁

06

가장 위험한 경쟁자

Notion이다. 직원이 이미 글을 쓰는 곳이고, AI가 Mermaid 다이어그램을 출력하며, DB 속성으로 담당자·도구·시간을 담고 롤업으로 합계를 낸다. 템플릿 한 장으로 이 제품의 70%가 재현된다. 부족한 건 아웃라인과 다이어그램의 양방향 동기화 하나뿐이다.

두 번째는 **Scribe** 같은 태스크 마이닝인데, 기능이 아니라 입력 비용에서 이긴다. 적지 않아도 기록되기 때문이다. 다만 브라우저 밖은 못 본다.

해자가 있는가 — 냉정하게

주장

판정

아웃라인 → 자동 다이어그램	해자 아님. Napkin·Whimsical AI·Notion이 유사 결과를 낸다. 4~8주면 복제된다. 다만 이들은 "한 번 생성"이라 3주 후 문서와 그림이 어긋난다 — 양방향 유지보수는 6~12개월 리드타임
-----------------	---

자동화 후보 스코어링	계산식은 3개월이면 복제된다. 복제 불가능한 건 축적된 데이터 — 전 직원의 단계·도구·시간 레코드, 도구명 정규화 사전, 부서 간 중복 단계 그래프
-------------	---

결론: 기능 해자는 없다. 데이터 자산 해자는 실재하나, 사용자가 꾸준히 입력할 때만 형성된다. 즉 이 제품의 방어력 전부가 "입력 습관 형성"이라는 한 점에 걸려 있다.

포지셔닝

두 갈래를 함께 쓴다. 유료 헤드라인은 "직원들이 직접 만드는, 우리 회사의 자동화 백로그", 무료 진입로는 "쓰기만 하면 흐름도가 되는 업무 기록 — 인수인계와 온보딩용"이다.

진입로만으로는 예산이 없고, 헤드라인만으로는 사용자가 없다.

요금 경계는 프라이버시 경계와 일부러 일치시켰다 — "자기 일에 대한 것은 전부 무료, 여러 사람 일을 합쳐 보는 것만 유료." 이러면 가격 협상이 정책 협상으로 번지지 않는다. "돈 더 내면 개인 데이터 볼 수 있나요?"라는 질문이 성립하지 않기 때문이다.

무엇을 어떤 순서로 만드나

07

증명할 단 하나의 가설

비개발자가 도움 없이 10분 안에 자기 업무 하나를 기록하고, 자동 생성된 다이어그램을 남에게 보여줄 만하다고 판단해 실제로 공유한다.

"공유했다"까지가 가설이다. 만들었다는 것만으로는 아무것도 증명되지 않는다

단계	범위
M0	디자인 토큰 · 스키마 · 스파이크 6건
M1	붙여넣기 파싱 → 질문 연쇄 → 아웃라인↔다이어그램 안정적 연동 · 저장 · 공유 링크
M1.5	인수인계 문서 · 휴가 대체자 가이드 내보내기. 직원의 이기적 이득이 여기서 처음 손에 잡힌다
M2	메타데이터 카드 스택 · 짜증 플래그 · 댓글
M3	요약 카드. 랭킹이 아니라 요약 — 사람들은 여기서 스스로 병목을 찾는다
M4	체크리스트 모드 · 핸드오프 조립 · 불일치 리포트

초안은 마일스톤을 개발 난이도 순으로 짰는데 잘못이었다. 직원이 자기 이득을 손에 쥐기 전까지는 메타데이터를 채울 이유가 없고, 메타데이터 없이는 분석이 무의미하다. 그래서 채택 순으로 재배열했다 — 저장과 공유가 M1으로 올라오고, 내보내기가 M1.5가 됐다.

일정에 대한 정직한 말

초안의 9.5주는 2인 기준 23.5주, 1인 기준 45주가 현실이다. 그리고 그 반증은 저장소 안에 있었다 — 컴포넌트 문서가 컴포넌트 16개를 만드는 데만 5주를 배정하는데, 초안의 M0 + M1 합계가 3.5주였다. 같은 저장소의 두 문서가 서로를 반증했다.

롤아웃 — 전사 동시 오픈은 하지 않는다

단계	내용
0 · 시드	도구를 아무에게도 주지 않는다. 팀이 직접 8~12개 부서를 인터뷰해서 대신 작성한다. 콜드 스타트는 직원이 아니라 팀이 채워서 푼다
1 · 파일럿	인사팀부터. 감시자로 의심받는 부서가 먼저 자기 것을 벗으면 "감시 도구" 프레임이 깨진다
2 · 계기 확산	홍보하지 않고 이벤트에 넣는다 — 인사이동 인수인계, 신입 온보딩, 감사 대응, 장기휴가. "이 도구 써보세요"가 아니라 "인수인계 이번엔 여기에 적어주세요"
3 · 챔피언	부서장이 아니라 질문을 가장 많이 받는 실무자 3~7년차. 임명하지 말고 로그에서 자연 발생한 사람을 지목한다

지금 상태 — 정직하게

08

기획 완료. 문서 26개 · 37,000줄

계산 계층 완료. 8패키지 · 32,000줄 · 테스트 784건 통과

앱 계층 없음. `.tsx` 0개, `.css` 0개

사람이 브라우저를 열어 쓸 수 있는 것은 아직 아무것도 없다. 지금 돌아가는 것은 계산 계층뿐이다 — 트리를 그래프로 파생하고, 붙여넣은 글을 단계로 쪼개고, 좌표를 계산하고, 인수인계 문서를 만들고, 자동화 후보를 점수화한다. 전부 함수 호출로만 확인된다.

구현이 기획을 반증한 것들

명세를 코드로 옮기니 문서만 쓸 때는 나오지 않는 결함이 드러났다. 지금까지 24건이고 전부 테스트로 고정돼 있다. 대표적인 셋:

데이터 발산 텍스트 3-way 병합이 양쪽이 같은 지점에 삽입하면 순서에 따라 다른 결과를 냈다. 충돌 표시 오류가 아니라 두 클라이언트가 서로 다른 문장에 수렴하는 문제였다

설계 원칙의 반례 "편집한 노드를 화면에 고정한다"가 항상 옳지는 않았다. 움직인 노드가 그것뿐일 때는 캔버스를 미는 보정이 오히려 20~33배 나뉘었다

산술 오류 시드 데이터의 자동화 후보 카드에서 공식과 결과가 달랐다 — 적힌 식대로 계산하면 378만인데 카드에는 270만이 인쇄돼 있었다

아직 안 정한 것

제품 이름 `Preflow` 가 유력하다. 도메인·로고·공유 페이지 워터마크가 여기 의존한다

n8n 앞 vs 옆 자동화 후보를 발견만 할지, 실행 가능한 명세까지 떨어뜨려 자동화팀에 위임할지. 파일럿 데이터로 결정한다

다음에 할 일

가장 얇은 수직 스켈레톤 하나. 로그인 → 한 줄 입력 → 노드 하나 → 공유 링크 → 다른 사람이 폰에서 열람.

지금 이 프로젝트의 가장 큰 미검증 리스크는 로직이 아니라 **접합부**다. 이 계산 계층이 실제 UI와 붙는지는 코드를 더 읽어서 안 풀리고, 붙여봐야 안다. 그리고 M1 가설 문장이 정확히 그 경로다 — 이 한 줄에 프로젝트 위험이 전부 한 번씩 걸려 있다.

이 문서가 만들어진 방식

사람 저자 한 명과 Claude가 함께 썼다. 21개의 서로 다른 **역할 관점**으로 나눠 검토를 반복했고, 각 관점은 서로의 결론을 모른 채 판단했다. 총돌 지점은 사람이 판정했으며 그 근거가 결정 기록 79건이다. 외부 전문가가 개별적으로 검토한 것은 아니다.

이 방식의 한계도 적어둔다 — 관점이 아무리 많아도 같은 모델의 같은 편향을 공유한다. 그래서 검증 가능한 것은 전부 코드로 옮겼다. 위 §8의 "구현이 기획을 반증한 것들"은 관점을 늘려서가 아니라 **실행해봐서** 나온 결과다.

전체 문서 26개와 코드는 저장소에 있다 · github.com/wlsdks/workflow-builder